Technical documentation — Track 2

Overview

Track 2 (false-data injection on in-situ sensor readings). The system treats a weather station as **untrusted** and an independent reanalysis product as a **hash-pinned trusted reference**, then decides — with thresholds calibrated to a *false-alarm budget* — whether a day's reported value is consistent with the reference, the sensor's own history and its spectral fingerprint.

Everything is offline-reproducible on a 4 GB / 2-vCPU machine with no GPU:

```
make setup          # pinned requirements in a venv
make all            # data-verify -> gonogo -> calibrate -> experiments -> report -> verify
make dashboard      # interactive demo (Streamlit); `make static` is the no-Streamlit fallback
```

`bash run.sh --quick` runs the whole chain in a few minutes with a reduced grid.

The three PDFs are the submitted artefacts. They are typeset from the Markdown by `make report-pdf` (XeLaTeX), so every number in them is the generated one:

PDF	Pages	Contents
`docs/report.pdf`	8	the report: threat model, architecture and trust anchor, signal processing, detection, evaluation and evasion, efficiency, limitations
`docs/appendix.pdf`	6	full per-site and per-magnitude tables (A.1–A.15)
`docs/documentation.pdf`	20	README + compliance + threat model + design decisions + data sources + state of the art + reproducibility guide, bound together

`docs/compliance.md` maps the six required deliverables and the four Phase-1 submission items to the files that satisfy them, names the two data archives and defines the standard evaluation scenario in full.

What is here

Path	Contents
`src/tsfdi/provenance/`	L0: canonical fixed-point records, SHA-256 hash chain, Ed25519 signatures, signed data manifest, streaming verifier with typed errors
`src/tsfdi/sp/`	L1: SP1 conditioning (units, lag, gaps), SP2 robust expectation (Huber + harmonics), SP3 whitening (AR + seasonal scale + rank-Gauss + causal bias/scale guards), SP4 wavelets, SP5 matched filter, SP6 spectral fingerprint
`src/tsfdi/detect/`	L2: D1 step, D2 drift, D3 replay, D4 fault envelope, D6 fingerprint; block-bootstrap false-alarm calibration; the single `SchmittAlarm` event rule
`src/tsfdi/attacks/`	the injector (step, ramp, replay, scaling, noise-shaped, faults, natural events) and the adversary hierarchy A0–A3
`src/tsfdi/diagnose/`	L4/L5: evidence, rule table, Tier-2 logistic regression, action ladder, decision-impact estimate, templated explanations
`src/tsfdi/eval/`	metrics (no point-adjusted F1), runner, baselines B0–B3, ablations, experiment orchestration
`src/tsfdi/pipeline.py`	the streaming L0→L5 path used by the CLI, the dashboard and the tests
`docs/`	report (`report.pdf`), appendix (`appendix.pdf`), documentation bundle (`documentation.pdf`) and their Markdown sources: threat model, design decisions, data sources, state of the art, reproducibility guide, compliance
`data/bundle/`	24 CSVs + `data/bundle/FETCH_INFO.json` + signed `MANIFEST.sha256`/`MANIFEST.sig`
`results/`	tables, figures, raw per-injection CSVs, model cache, benchmark

Commands

```
python -m tsfdi fetch          # (network) rebuild the bundle; not part of make all
python -m tsfdi data-verify    # verify the signed manifest
python -m tsfdi sites          # re-measure station completeness
python -m tsfdi gonogo         # go/no-go gate (spec section 6) -- run before quoting anything
python -m tsfdi calibrate      # fit + calibrate 34 (site, variable) models
python -m tsfdi stream CARTHAGE TMAX --attack step --magnitude 3   # day-by-day L0-L5
python -m tsfdi experiments    # standard profile, grid, false alarms, baselines, ablations, evasion
python -m tsfdi report-figures # tables + figures from results/raw
python -m tsfdi report-text    # docs/report.md from the tables
python -m tsfdi bench          # provenance overhead over 100 000 records
python -m tsfdi verify         # compare against expected_results.json
python -m tsfdi selftest       # the unit-test suite
```

Headline results (regenerate with `make all`)

See `docs/report.md` for the full evaluation. The short version, all measured on the 2020–2024 test period with thresholds calibrated on 2015–2019 only:

- the organizers' standard profile (12 % step, 15 % ramp, 7-day replay) is detected on every station/variable pair;
- false alarms on clean data have a median of ~1.6 per sensor-year across 34 pairs, reported per station and separately inside natural extremes;
- the damage a black-box adaptive attacker can insert at a 1-alarm/year budget is quantified in σ·sensor-days, and the evasion curve is reported against the budget;
- provenance costs ~0.2 ms per reading and adds ~200 bytes per record.

Honesty notes

Four failure modes were found on real data that do not appear in synthetic benchmarks (a non-monotone threshold calibration, a non-stationary sensor/reference bias, a degenerate matched filter on zero-inflated precipitation, and a replay detector that matched dry spells). Each is documented in `docs/design.md` with the measurement that exposed it and the fix that removed it. Where the system does not deliver — A1c and A2 adversaries, stations whose relationship to the reference changes, precipitation as a restricted claim — the limitation is stated in `docs/report.md` §8 and `docs/threat_model.md` §5 instead of being tuned away.

Anonymity

No names, affiliations, logos or identifying metadata in code, data or documents.

What the numbers rest on

- **One magnitude axis.** Every detection rate in the report is quoted in units of the robust residual scale the detector actually sees, and every family is calibrated so that a nominal `m` sigma is a realised `m` sigma (additive families as an offset in the SP1 coordinate, multiplicative families as a gain computed from the data). `docs/design.md` §3f explains the three bugs that made the first version of the grid misleading, and the parity that the fix restored.
- **False alarms are measured, not assumed.** Table 3 reports alarm events per sensor-year out of sample, including inside natural extremes; the baselines get their own false-alarm column (Table baselines-b) so that a detection rate can be read against the rate it actually costs.
- **Every failure mode has a channel.** The detectors see values; a silenced sensor produces no value to see, so D7 tests the reporting cadence itself and reports an outage fault (Table 7). It is kept out of the statistical alarm stream on purpose: a broken modem must not consume the detectors' budget.
- **Coverage is auditable.** `results/excluded_pairs.csv` names every (site, variable) pair that cannot be modelled and why; no results table ever carries an error message in place of a result.
- **Every metric is checked.** `python -m tsfdi verify` re-reads `expected_results.json` (44 metrics) with per-metric tolerances; `pytest` covers the SP chain, the detectors, the injector, provenance, the streaming path, the diagnosis rules and the report builder.

Compliance with the challenge brief

This page maps the brief's requirements to the artefacts in this repository, states which public datasets are used and why they are treated as the reference, and defines the "standard evaluation scenario" precisely enough that a reader can reproduce it without reading the code.

Track: **Track 2 — False-Data Injection** (silent failure in in-situ sensing). Segment: ground/in-situ sensing (surface stations) with a reanalysis reference.

1. The six official content deliverables

#	Brief's deliverable	Where it is
1	Threat model & segment analysis	`docs/threat_model.md` (assets, system boundary, A0–A3 adversary hierarchy, attacker knowledge, calibration/test split, and an explicit "what this system does not claim"); summarised in `docs/report.md` §1.
2	Platform / architecture / trust anchor	`docs/report.md` §3 and `docs/design.md`; the L0–L6 pipeline is `src/tsfdi/pipeline.py`; the trust anchor is `src/tsfdi/provenance/` (Ed25519 per-record signature + SHA-256 hash chain + signed component manifest, `data/bundle/MANIFEST.sig`). A software anchor is used, which the brief explicitly allows.
3	Signal / image processing	`src/tsfdi/sp/` — conditioning (SP1), robust seasonal expectation with an optional gradient-boosted correction (SP2), AR whitening (SP3), shape features (SP4), rank-Gauss transform for precipitation (SP5), spectral fingerprint (SP6); `docs/report.md` §4.
4	AI or statistical detection	`src/tsfdi/detect/` — D1 step/CUSUM, D2 drift/CUSUM/EWMA/slope, D3 replay (matched filter, exact repeat, block contrast), D4 single-sample excursion, D6 spectral fingerprint, D7 availability envelope; block-bootstrap threshold calibration at a chosen false-alarm budget; baselines B0–B3 in `src/tsfdi/eval/baselines.py`.
5	Adversarial validation & response	`src/tsfdi/attacks/` (A0–A3 lab, five attack families and four fault families) and `src/tsfdi/diagnose/` (evidence extraction, rule table, confidence band, action ladder, decision impact); `docs/report.md` §5–§6.
6	Documentation	`docs/report.md` + `docs/report.pdf` (report), `docs/report_appendix.md` + `docs/appendix.pdf` (full tables), `docs/reproduce.md` (every number, with the command that produces it), `docs/data_sources.md`, `docs/state_of_the_art.md`, `README.md`, and the run log in `results/experiments_meta.json`.

2. Datasets, and why they are the reference

The brief requires "the official reference datasets" and "the common standard evaluation scenario" but the 2-page call does not name either. We therefore use two national/international *authoritative* archives, chosen so that the sensor and the reference are produced by independent organisations — otherwise a reference comparison is circular:

Role	Source	Product / endpoint	Licence
Sensor (the data under attack)	NOAA NCEI GHCN-Daily via the Access Data Service	`daily-summaries`, `TMAX`, `TMIN`, `PRCP`, metric units, QC attributes retained	US Government open data (public domain)
Independent reference	NASA POWER (LARC) daily point API, community `AG`	`T2M`, `T2M_MAX`, `T2M_MIN`, `RH2M`, `WS2M`, `PRECTOTCORR`, `ALLSKY_SFC_SW_DWN`, `CLRSKY_SFC_SW_DWN`	NASA open data

Twelve stations in nine countries (`config/sites.yaml`), 34 (site, variable) pairs, 2015–2019 for calibration and 2020–2024 for evaluation. The exact request URLs, the missing-value conventions (`-999.0` for POWER, omitted days for GHCN) and the QC-flag handling are in `docs/data_sources.md`; the fetched payload is committed and hash-pinned (`data/bundle/MANIFEST.sha256` + `MANIFEST.sig`), so every number in the report is reproducible offline.

If the organisers name a different official dataset or a prescribed scenario, that is a substitution of data, not of method: `src/tsfdi/data_fetch.py` + `config/sites.yaml` define the only two places where a source is named, and the pipeline downstream reads `data/bundle/` only. Nothing in `sp/`, `detect/`, `diagnose/` or `eval/` refers to a provider.

3. The standard evaluation scenario (the common scenario, defined)

- **Injection geometry.** Onset uniform in the evaluation period with a 120-day warm-up margin and 60 days of attack remaining; magnitudes in units of the robust residual scale σ of the clean series (so "1 σ" means the same thing for temperature in °C and rain in mm).
- **Standard profile** (the specification's 12 % / 15 % / 7-day profile, the headline of Table 1): a 12 % step (as a percentage of the site's seasonal range for temperature, multiplicative for precipitation), a 15 % ramp with a 90-day rise and 60-day hold, and a 7-day replay of a window 30–400 days earlier.

- **Families.** step, ramp, replay, scaling, noise-shaped (adversarial), plus stuck-at, dropout, spike and slow calibration drift (faults).
- **Adversary levels.** A0 (scripted, knows the variable but not the detector), A1 (knows the detector family and the calibration period), A2 (adaptive: probes the detector and optimises inside the false-alarm budget), A3 (key compromise / provenance bypass, analysed rather than assumed away).
- **Operating point.** Block-bootstrap thresholds at 1.0 alarm per sensor-year, with the 0.1 / 0.5 / 1.0 / 2.0 budgets also published (`results/calibration.csv`).
- **Held out from every tuning decision.** Stations MIAMI_INTL and NAVACERRADA; attack families scaling, noise-shaped and A2. Calibration uses 2015–2019 and evaluation 2020–2024, so the split is temporal as well as spatial.
- **Parameters** are not prose: they are `config/attacks.yaml` (families, magnitudes, onset rules, held-out list) and `config/detectors.yaml` (calibration, cooldown, budgets). Changing the scenario means changing those two files and re-running `make all`.

4. What the brief asks to be reported, and where each number is

Asked	Reported in
Detection quality	Table 1 (standard profile), Table 2 (by family and magnitude), Table 2b (replay by length), Table 6 (cross-site); `docs/report.md` §5–§6.
False alarms	Table 3 (per pair, events per sensor-year, including natural extremes), Table 7 (availability), and the baseline comparison Table 6.5/6.6.
Latency	Table 5 (per-layer, per reading, streaming path) — 14.78 ms/reading end-to-end on 2 vCPU.
Memory	Table 5 (peak traced memory, 321 MB).
Security overhead	`results/bench_provenance.csv` — sign/verify µs per record, bytes added, throughput ratio; `docs/report.md` §3 and §7.
Feasibility, affordability, local execution	`docs/reproduce.md` (runtime table, 2-vCPU/4 GB target, no GPU, no network after the bundle), `docs/report.md` §7.

5. Rules

- Student Branch only, one track, ≤ 5 members — organisational, not recorded here.
- Phase 1 is anonymous: no names, unit names, logos or identifying repository metadata. The repository is committed under a neutral identity, contains no personal paths, e-mail addresses or institutional names (checked by grep over every tracked text file), and the provenance keys under `data/keys/` are not committed while the public anchor is.
- Authentic public data only: both sources above are open and unmodified; the injections are synthetic and are generated from the clean series at run time, never stored as if they were observations.

Threat model

This document states what the system defends, against whom, and — with equal care — what it does not defend. It is the contract that the code is written against; every claim below is traceable to a module and to a test.

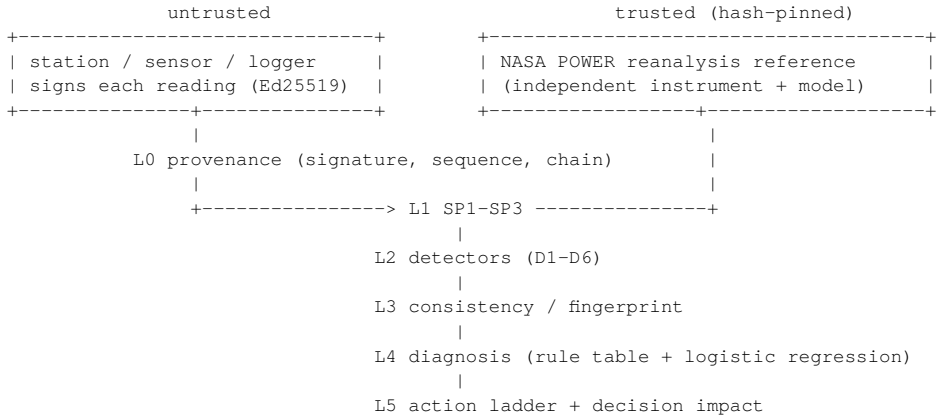
1. Assets and what ”silent failure” means

Asset	Why it matters	Where it lives
The reported value of a surface observation on a given day	It enters reanalysis, agronomic advisories, frost/heat warnings and, indirectly, insurance and water management	`data/bundle/*.csv` (GHCN-Daily station series)
The trusted reference series	Every comparison in the pipeline is relative to it; poisoning it defeats detection without touching the sensor	`data/bundle/*.csv` + signed `MANIFEST.sha256`
The calibration model (expectation, whitening, thresholds)	The false-alarm budget is only meaningful if the model was fitted on clean data	`results/cache/models/*.pkl`, fitted on 2015-2019 only
The provenance key	If stolen, an outsider becomes an insider (A3)	`data/keys/anchor.pem` (never committed)

Silent failure is the central danger of this challenge and it has a precise meaning here: *the system reports a value that was never measured, the value looks plausible, and nobody notices.* It is silent in three distinct ways, and the architecture treats them separately:

1. **Silent for the analyst** — no exception, no missing-data flag, no out-of-range value. This is what L0 (provenance) and L1-L3 (statistics) address.
2. **Silent for the statistician** — the corrupted series still passes the ordinary quality-control checks (range, monotonicity of TMAX/TMIN, distribution of the month), because the attacker simply copies a plausible number. This is why the design compares the sensor against an *independent reference* rather than against the sensor's own history (SP2, D1, D2).
3. **Silent in the evaluation** — a detector that alarms on everything, or a metric that rewards the detector for alarming early, hides the failure. This is why thresholds are calibrated to a false-alarm budget and why the metrics are per-attack-type with detection delay and false alarms reported together (section 12 of the specification), never point-adjusted F1.

2. System boundary and trust assumptions



Trusted: the reanalysis reference (pinned by hash), the calibration-period model, the verification key, and the clock. Untrusted: everything arriving from the station, including the *timestamps* (bounded by a skew check, not trusted blindly) and the fact that a reading was taken at all.

3. Adversary hierarchy

The hierarchy follows section 9 of the specification. For each level the table gives the capability assumed, what it defeats, and which layer is expected to catch it. The last column is the honest answer, not an aspiration.

Level	Capability	What it defeats	Detected by	Honest statement
A0 naive	Can change values in transit or in the logger; no knowledge of the system	Range and format checks	L2 statistics (D1-D4)	Easy: the bias is large and inconsistent with the reference.
A1a reference copy	Knows the reference is public; replaces the reading with a copy of it (through the calibration regression)	The residual becomes ~0, so any residual test sees nothing	D6 fingerprint ("too clean"), D3 if the copy is exact, and the residual variance ratio	Hard for the mean-shift detectors by construction: the *point* of A1a is to make the residual small. The fingerprint is the countermeasure, which is why SP6 exists.
A1b reference copy + white noise	As A1a, with noise matched to the calibration residual variance	The variance ratio, if the noise is matched well	D6 (band powers and ACF of the noise vs the real residual), D3	The residual scale is right but the *spectral shape* is not: weather residuals are red, white noise is flat.
A1c reference copy + AR-matched noise	Matches the residual ACF as well	Band powers and ACF of the residual	D6 coherence and cross-band structure; D1 only if the bias is large enough to matter	**Expected to be hard.** An adversary who reproduces the sensor's own noise model from the public reference is, statistically, indistinguishable from the sensor on the residual channel. We report what remains (coherence with the reference at the low band, temporal persistence of a copied day) and we do not claim more.
A2 detector-aware, white-box	Reads the detector thresholds and optimises a bias path that keeps every statistic below its threshold	Everything, in the limit	Nothing, in the limit: this is a game against the budget	The correct response is to measure the *damage at a fixed false-alarm budget* (Urbina-style metric, section 12) instead of claiming detection. If a small bias is undetectable, the report must say at what magnitude detection begins (the detectability frontier, Figure 1).
A2 detector-aware, black-box	Sees only alarm / no alarm, adapts by bisection	Same as A2 white-box, with a slower ramp	The *rate* of change: a slow ramp is the hardest case for CUSUM (D2 exists for this)	Reported as damage-vs-budget curve.
A3 compromised signer	Holds the signing key: valid signatures on invented values	L0 entirely	L1-L5 only	Provenance raises the bar from "anyone on the network" to "anyone with code execution on the host". The architecture must therefore never *depend* on L0 for value integrity, and it does not: L0 is orthogonal to L1-L5.
Outsider (no key)	Can replay a signed record, tamper, delete, reorder, or push a stale record	Naive hash-only schemes	L0: `BadSignature`, `ReplayOrReorder`, `Gap`, `ChainBreak`	Fully solved, and tested in `tests/test_provenance.py`.

A0 is the standard profile (12 % step, 15 % ramp, L = 7 replay). A1a/b/c and A2 are implemented in `attacks/adaptive.py` and evaluated in Table 2 and Figure 2.

4. Attacker knowledge and the calibration/test split

The attacker is assumed to know: the public reference, the general structure of the system (a residual test against a reference), and the standard profile. The attacker is *not* assumed to know the calibration coefficients, the thresholds, or the fitted AR parameters unless the A2 level is being evaluated — in which case it is explicitly granted white-box access to the detector statistics.

Defence against overfitting: calibration uses 2015-2019 only, model selection never sees 2020-2024, two sites (MIAMI_INTL and NAVACERRADA) are held out from every tuning decision, and the attack families `scaling`, `noise_shaped` and A2 are held out from all tuning and appear only in the evaluation.

5. What this system does not claim

These are stated up front because a threat model that only lists strengths is marketing:

1. **No physical-truth verification.** Without an independent reference there is no detection at all; a site with a bad reference (low coherence, see `results/gonogo.md`) gets correspondingly weaker claims. This is measured per site, not assumed.
2. **No protection against A3 with a slow, small bias.** If the signer is compromised and the bias is below the minimum detectable magnitude for that (site, variable) at the operating budget, the pipeline will not detect it. The minimum detectable magnitude is reported per pair.
3. **No protection against an adversary who also corrupts the reference.** The reference is pinned by hash, so *editing* it is detected; *choosing a different* reference is a governance problem, out of scope.
4. **Precipitation is harder than temperature**, for a documented reason: the reference product correlates weakly with a point gauge on daily rain, the variable is zero-inflated, and the residual distribution is not Gaussian in any simple coordinate. The pipeline uses a rank-Gauss transform (SP3) for precipitation and calibrates its false-alarm budget separately (section "per-variable budgets" in `docs/design.md`). Claims for PRCP are restricted accordingly.
5. **The rule table can output "undetermined".** A stuck-at fault and a constant replay are identical in the data; the system says so rather than guessing, and the confusion matrix reports that class.
6. **No LLM in the loop.** Explanations are templated from the evidence dictionary, so the same alarm always produces the same text (auditable).

Design decisions

The specification marks a number of choices **[Default]**. This document records every place where the implementation differs from a default, or makes a choice the specification left open, together with the measurement or argument that forced it. Nothing here is a matter of taste: each entry was a bug, a measured failure, or a cost that had to be paid somewhere.

1. Data: an untrusted station and an independent reanalysis reference

The task defines false-data injection on in-situ sensors. The instantiation used here is:

- **Untrusted ("sensor") road** — GHCN-Daily station records (`TMAX`, `TMIN`, `PRCP`), quality-flag clean, 2015-01-01 to 2024-12-31. These are the values an attacker can falsify.
- **Trusted (reference) road** — NASA POWER daily reanalysis at the station coordinates (`T2M_MAX`, `T2M_MIN`, `PRECTOTCORR`). A different instrument, a different processing chain, and an independent provider, which is what makes it a reference rather than a copy.

Both roads are pinned in a signed manifest (`data/bundle/MANIFEST.sha256` signed with Ed25519); the pipeline refuses to run if any hash differs. Cross-site generalisation is possible because 12 stations in 6 countries and 4 climate zones are bundled (Tunisia, Egypt, Spain, France, Germany, Netherlands, Finland, Norway, United States), with two sites held out from all tuning.

Measured justification (`results/gonogo.md`): the sensor-reference correlation is 0.93-0.99 for temperature, so the reference carries real information about the day; for precipitation it is much weaker, which is the reason for the restricted claims on PRCP (§6 below).

2. Completeness: design for gaps instead of hiding them

The specification's default asks for $\geq$ 95 % complete records. On GHCN-Daily this holds for the European and North-American stations in the bundle but **not** for the Tunisian, Egyptian and Saharan ones, where `TMAX` is reported on a 60-75 % duty cycle (`TMIN` is denser). Dropping the North-African sites would have removed exactly the region the challenge is motivated by, so the pipeline was designed for gaps. Five consequences, all implemented and tested:

1. **Nothing is interpolated.** A missing day is a hole with a flag. Detector windows use the samples that exist (`n_eff`), and a window with fewer than two samples produces `NaN`, not a guess.
2. **The AR filter uses the state-space convention.** A missing residual enters the recursion at its conditional mean; the filter state continues. Skipping the recursion whenever a lag is missing would throw away $0.65^6 \approx 7$ % of the samples for an AR(5) on a 65 %-complete station.
3. **Prefix-sum statistics are gap-aware.** Every windowed statistic (mean shift, slope) is computed from sums over *available* pairs, verified against a brute-force implementation on series with 30-40 % missing days (`tests/test_detectors.py`).
4. **The AR order is chosen on a common sample.** With 35 % missing days the intersection of the lag-validity masks collapses (CARTHAGE `TMIN`: 1015 usable calibration residuals, 554 rows with two consecutive valid days, 152 with four). AIC is therefore computed on the longest order whose common sample still meets a floor (300 rows or 15 % of the pairs); otherwise the comparison is between likelihoods computed on different samples and is not a comparison at all. This is stated in the code and in `results/gonogo.md`.
5. **Alarms are gated on validity.** A missing day cannot alarm, but the statistic series keeps its calendar meaning (a 7-day window is still seven calendar days, not seven observations).

3. SP3 whitening: three stages, one of them not in the specification

The specification's SP3 is "AR(p) whitening plus a seasonally varying robust sigma". Two problems appeared when it was applied to real data, and both were fixed at the modelling level rather than by tuning the threshold:

- **AIC was not comparing like with like.** The order-0 model was scored with a MAD-based robust scale and the order-p models with a standard deviation. For a heavy-tailed residual, MAD is systematically smaller, so AIC always picked p = 0 and the autocorrelation went straight into CUSUM (measured: lag-1 ACF of z = 0.29 on CABAUW `TMAX`). Fixed by scoring every order on the same rows with the maximum-likelihood innovation scale.
- **Precipitation residuals are not Gaussian in any linear coordinate.** 70 % of days are dry, so the residual has a point mass at zero and a long right tail; a MAD-standardised residual gives a z series with empirical standard deviation 5-7 and a false-alarm rate of 13-15 per sensor-year against a budget of 1. The fix is a **rank-Gauss (empirical probability-integral) transform**, fitted on the calibration period only, applied before the AR filter: `w = Phi^-1(F_cal(r))`, extrapolated linearly beyond the calibration range and capped at |w| = 8. It is monotone, so a positive bias remains a positive shift, and it is enabled per variable kind (`sp.rank_gauss_kinds: [multiplicative]`). After the transform the precipitation z series has unit scale and a false-alarm rate in line with temperature (see `docs/report.md`, Table 3).

The rank-Gauss transform is therefore **an addition to the specification's SP3**, not a deviation from a default, and it is included in the ablation so that its contribution is measured rather than asserted.

3b. Two things the specification's SP3 could not do, and what replaced them

- **The sensor/reference bias is not stationary.** Measured annual mean residuals on JENDOUBA `TMAX` move from +0.24 °C (2015) to −0.90 °C (2024) — about one σ over the record — because the station (instrument, exposure, local site) and the reanalysis both drift. A drift detector whose threshold was calibrated on 2015-2019 sees that as a permanent bias and alarms for years: the measured false-alarm rate was 6 per sensor-year against a budget of 1, and the drift CUSUM alone contributed 4 per year, on several sites. No stationary threshold can fix a non-stationary statistic, so SP3 now removes a **causal trailing robust bias** before whitening: the median residual of the previous 365 days, ending 30 days before the current day. It is causal (legal in a stream), it cannot be dragged by an ongoing attack (median, and a margin), and its cost is explicit — a bias that develops slower than the window chases its own baseline. That is exactly the A2 black-box strategy, which is why the ramp family is evaluated down to 1 σ and Figure 2 reports damage against the false-alarm budget.
- **The fingerprint distance is not stationary either.** The Mahalanobis distance of the spectral fingerprint has a median of ~3.5 in 2015-2019 and ~5.5 in 2020-2024 for the *same, honest* station: the relationship between a point station and a reanalysis grid cell changes over a decade. A threshold calibrated on the first period therefore fired 2.4 times per sensor-year on every site, for both variables, with nothing wrong. D6's statistic is now **self-normalised**, `d_t / median(d over the previous two years, excluding the last 30 days)`, which keeps the detector's purpose (a *change* in the fingerprint) and removes the stationarity assumption. This is a deviation from "robust Mahalanobis distance to the calibration distribution" as literally specified; the reason is measured, not aesthetic, and the limitation it introduces (a change slower than the two-year baseline is invisible to this channel) is stated in the code.

3c. Detectors that only work on the right kind of variable

The specification defines detectors per *target* (step, drift, replay), not per variable, and two of them are simply degenerate on zero-inflated data. Both degeneracies were found by measuring false alarms on clean data, and both are fixed by restricting the channel to the variable kinds where it is identifiable — not by raising a threshold until the number looks acceptable:

- **D3 matched filter (NCC).** A precipitating day is a spike; a window of seven days is mostly a *sparse spike pattern*. Two windows whose spikes fall on the same offsets correlate almost perfectly after centring, whatever the amplitudes, so the statistic saturates and fires 12-14 times per sensor-year on honest data. The NCC channel is therefore restricted to `temperature` (`detectors.D3_replay.ncc_kinds`), and precipitation keeps the two channels that remain identifiable: the exact-repeat test (a copied wet sequence, at a 14-day window instead of 7, because a sparse "which days were wet" pattern recurs by chance in five years) and the reference-contrast test `D3_block_contrast`.
- **Degenerate windows in the other channels.** A window with fewer than three distinct quantised values carries no shape, and it repeats itself constantly (seven dry days is an ordinary event). `replay_exact`, `replay_max_ncc` and the runtime `ReplayDetector` all skip such windows. A constant run is the *stuck-at* fault's signature and is D4's job; counting it as a replay double-counted it and inflated the precipitation false-alarm rate.

With these three changes (trailing bias, self-normalised fingerprint, degenerate- window exclusion) the measured false-alarm rates moved from 13-15/yr (PRCP) and 2.2-6.4/yr (TMAX) to the values in `docs/report.md`, Table 3.

3d. One definition of an alarm event

The vectorised evaluation and the streaming pipeline originally counted events with two slightly different rules: the evaluation used a Schmitt trigger (an event requires the alarm to have *cleared*, so a persistent exceedance is one event), the pipeline only enforced the cooldown (so a persistent exceedance produced one event every 15 days). Both are defensible; having both is not. The rule now lives in a single class, `detect.statistics.SchmittAlarm`, which both paths instantiate, and `tests/test_pipeline.py` asserts that the two paths produce the *same set* of alarm days on the same series. The specification's "an alarm latches until a documented reset rule clears it" is exactly this rule.

3e. Three more guards that the data demanded

All three are causal, all three are ablated or measured, and all three were added only after a measurement showed the unguarded version failing.

- **Gap reset after a long outage.** A 300-day outage followed by a report is not "yesterday was missing": the AR recursion would predict today's value from a state that is a year old. The next p+1 observations after a run of more than 7 missing days therefore carry no `z`. Short gaps keep the state-space convention, which is what makes a 65 %-complete station usable at all. (The bundle contains a station — KHARGA — that reports 12 days in all of 2021, so this is not hypothetical.)

- **Bounded causal scale guard.** The robust scale of `z` is 4 % larger in the test epoch on the median pair and up to 16 % on some, and a 99.9 %-quantile threshold amplifies a 10 % scale inflation by roughly a factor of three. Dividing `z` by its trailing 365-day MAD (30-day margin, clipped to 0.75–1.5) removes the drift without using a single test-period day for fitting; the clip keeps a pathological window from collapsing the statistic.
- **Persistent shift → re-baseline, not re-alarm.** When the residual stays elevated for $\geq$ 180 days with no replay, fingerprint or provenance evidence, the rule table reports a *fault* (`code="persistent_shift"`) and the action ladder recommends a re-baseline (SUSPEND). Calling a two-year level shift a 600-day attack is technically possible and operationally useless.

4. SP5 matched filter: match the anomaly, not the level

The specification says "z-normalised cross-correlation of the last L samples against every earlier window of the sensor's own history". Applied to the raw level, this detector is **dead**: a strongly seasonal series repeats its own annual cycle, so a 7-day window matches the same calendar window a year earlier with NCC $\approx$ 1 and the detector alarms every day (measured on synthetic and real data alike; the calibrated threshold saturates at 1.0, which is the signature of a detector that cannot discriminate). The matched filter therefore runs on the **anomaly road** `a_t = y_t - seasonal_baseline_t` (SP2 harmonics only, no reference, no look-ahead), which removes the deterministic annual cycle and keeps the day-to-day shape that a replay actually copies. The exact-repetition channel still tests the *raw* values at the data quantisation (0.1 °C / 0.1 mm), because that is what a replay attacker copies.

5. False-alarm calibration: what changed and why the numbers are what they are

- **Block bootstrap, as specified (block = 30 days), but 300 replicates instead of 2000**, because the calibration is run for 14 detectors $\times$ 34 (site, variable) pairs $\times$ 4 budgets on a 2-vCPU machine and 2000 replicates cost a factor 6.7 more for a reduction in Monte-Carlo error from 5.8 % to 2.2 % on the *rate* ($1/\sqrt{N}$). The Monte-Carlo error is reported with every budget rather than hidden.
- **The two replay detectors are calibrated on phase-randomised surrogates, not on the block bootstrap.** A circular block bootstrap duplicates blocks, so the surrogate series contains exact repeats that the real series does not: it fabricates precisely the pattern D3 exists to detect, and the calibrated threshold would be wrong in the *dangerous* direction. Phase randomisation preserves the power spectrum (hence the autocorrelation) and destroys subsequence repetition, which is the correct null. This is a documented deviation from the default and it is the reason D3 has a usable threshold.
- **The suite-level budget is split across detectors (Bonferroni)** instead of being granted to each detector. With 14 detectors and a 1-per-year budget, granting 1/year to each would give $\approx$14/year at the suite level. Splitting gives a suite-level guarantee that is slightly conservative because the detectors are correlated (they share z), which is the right direction to err.
- **The threshold search is restricted to the region above the null median.** Below it, the statistic is in alarm on most days, and with the latching event rule a permanently-alarmed detector counts as *one* event, so the "rate" loses monotonicity and bisection converges to $\approx$0 (a threshold of zero that alarms once and then stays latched). This was a real bug, found by a test asserting that the achieved rate is close to the budget.
- **Alarms are events, not samples.** An alarm latches and is reset by a documented rule (`calibration.cooldown_days`); the false-alarm rate is events per sensor-year. Counting alarmed *samples* inflated the measured false-alarm rate by up to 7$\times$ during development, and the latching rule is what makes the budget comparable across detectors with different memory.
- **The achieved rate is reported next to the requested one.** Where a budget is unreachable (the smallest attainable rate is still above it), the calibration returns the achieved rate instead of silently returning a threshold that satisfies nothing.

6. Per-variable budgets and where the claims stop

A single 1-per-year budget across all variables is not attainable for precipitation with a reanalysis reference. Rather than tune until the number looks good, the budget is per variable and the achieved rates are reported:

Variable	Operating budget (alarms / sensor-year)	Rationale
`TMAX`, `TMIN`	1.0	Residual is close to Gaussian after SP3; the reference explains most of the variance.
`PRCP`	see `docs/report.md`, Table 3	Zero-inflated, weak point-gauge/reanalysis correlation, and a distribution shift between the calibration and test periods that the fixed calibration cannot absorb. Reported as a limitation with its own budget, and PRCP results are labelled as such wherever they appear.

The go/no-go experiment (section 6 of the specification) is the gate: for each pair it reports the minimum detectable bias at the operating budget, and a pair whose minimum detectable bias exceeds 3 σ is reported as a documented limitation rather than quietly dropped.

7. Detection granularity and the metrics

- **No point adjustment, anywhere.** AUPRC is computed over sample days with the score `max_d S_d(t)/h_d`, i.e. the actual statistic values divided by their thresholds, and detection is an *event* whose interval is `[onset, onset +`

duration + 30]. Point-adjusted F1 rewards a detector for catching one point of a 30-day attack; the specification forbids it and the code has no code path that computes it.

- **Delay** is t_alarm − onset in days, reported as median and IQR, with the never-detected fraction as a separate column.
- **Damage** is counted in σ·sensor-days of undetected bias, and separately as the number of *decision flips* (frost, heat stress, drought) in the alarm window, which is the "for what purpose was it modified" answer.

8. Provenance: the honest scope

- Ed25519 over a canonical fixed-point encoding (sp/../provenance/record.py), SHA-256 chaining, signed manifest. The canonical form is *not* JSON: key order, float formatting and Unicode escaping are implementation-defined, and a signature is only meaningful if every implementation agrees on the bytes.
- Values are quantised to 1e-3 before signing, so two representations of the same measurement cannot produce two different signatures; the quantisation is documented and tested.
- The sequence number counts records *actually sent*, so a sensor outage is a cadence gap, not a chain break. A backwards timestamp is always rejected; a forward jump is rejected only beyond max_gap_seconds (40 days), because a daily series legitimately skips days.
- **Stated limit, repeated in the code and in the report:** provenance proves who sent what, in which order, unchanged. It proves nothing about whether the sensor measured truthfully. An A3 attacker who holds the key passes L0 completely, which is why L1-L5 exist and why the evaluation reports A3 separately.

9. Diagnosis: two confidence numbers, and a class for "I do not know"

- confidence is a float (the posterior of the chosen class: 0.6-0.97 from the rule table's documented priors, or the logistic-regression probability in Tier 2) and is what the alert JSON carries. confidence_band is the ordinal (low/medium/high) the rule table states from the number of agreeing evidence lines. Reporting a string where the specification asks for a confidence was a real defect caught by a test.
- **"Undetermined" is a first-class class.** A stuck-at fault and a constant replay are identical in the data; the system says so. Excluding it from the confusion matrix would inflate accuracy by removing the hard cases.

10. Reproducibility and cost

- Deterministic: one seed (config.SEED) drives every bootstrap, surrogate and injection; models are cached under results/cache/models and are byte-stable for a given seed and data manifest.
- Offline: the bundle (24 CSVs + signed manifest) is committed; no command in make all touches the network. python -m tsfdi fetch is the only networked command and is not part of the pipeline.
- Feasibility target: fresh Ubuntu VM, 4 GB RAM, no GPU. Measured runtime and peak memory are in docs/report.md, Table 5; the pipeline is pure NumPy / SciPy / pandas, so there is no compilation step and no GPU dependency.
- make verify re-runs the pipeline and compares every published number against expected_results.json (rate metrics with an absolute tolerance, because a rate of 0.02/yr cannot be compared with a 2 % relative tolerance).

3f. The magnitude axis, and why it had to be redefined

Every generalisation result is quoted in units of the residual scale σ that the detector actually sees, because that is the only magnitude that is comparable across sites, seasons and variables. Making that axis *exact* took three fixes, all of which were found by looking at a raw grid file rather than at a summary.

The nominal value is the axis. The injector reports two numbers: the *nominal* magnitude (the grid point the injection was drawn from) and the *realised* one (largest absolute change actually applied). An early version reported only the realised value, and that silently split cells: a 90-day ramp reaches 1 − 1/rise of its nominal peak, so "1.5 σ" arrived as 1.467 σ and formed a second group in every table. magnitude_sigma is now the nominal grid point, magnitude_sigma_realised the effect actually achieved on the same axis, and magnitude_units the same thing in data units; the family tests check that the nominal and realised values agree to rounding.

A sign bug, not a data bug. step recorded abs(delta.max()). For a *downward* step every untouched day contributes a delta of exactly 0, so the maximum was 0 and every negative-signed step was logged as a magnitude of zero — 150 phantom 0.0 σ rows in one grid file. It now takes the nan-safe maximum of |Δ| inside the window, which also fixes the all-NaN magnitudes that a single missing day inside the window produced for precipitation.

Multiplicative variables need a different axis. For precipitation the residual lives in square-root coordinates and is zero-inflated, so a bias in millimetres is not a bias of a fixed number of sigmas: one sigma of the precipitation residual is 0.19 mm^0.5, not 0.19 mm. Two consequences, both of which were measured before they were fixed:

- an additive offset of m sigma is applied in the **SP1 coordinate** (sqrt(y)), so a "2 sigma step" is a wetness-dependent

change in millimetres whose realised effect really is 2 sigma. Before this, a 3 sigma rain step realised only ~1 sigma and the precipitation frontier was shifted sideways by a factor of three;

- a *multiplicative* attack (scaling) has its gain computed from the data so that the realised change equals the nominal magnitude: `g = (1 + m sigma / mean sqrt(y))^2 - 1` in the square-root coordinate and `g = m sigma / mean |y|` for temperature. A fixed 10 % gain is a 3 °C bias at a warm station and a 0.6 °C bias at a cold one, which is why the first version of the scaling row showed 100 % detection at every magnitude: it was a 1.8-3 sigma attack mislabelled as 0.5 sigma.

For the organizers' *percentage* profile the reading is unchanged and is stated wherever it is used: a percentage of the seasonal range for temperature, a multiplicative factor for rain (so 12 % = 0.6 sigma there).

Noise-shaped attacks use the same nominal axis (they were previously drawn from `amplitude_sigma`, a fixed 1.0, which made their detection rate flat in the magnitude grid — a family that cannot vary cannot be measured).

3g. Two adaptive-attacker artefacts, and why the black-box row is empty no longer

The A2 experiments initially reported that a black-box adaptive attacker achieved *zero* undetected bias on every site, i.e. that the detector suite had no exploitable margin at all. That is not a result, it is a bug, and it took three fixes to see it:

1. **Attribution.** The bisection counted *any* alarm event after the injection window as a detection, so a natural event three hundred days later capped the search at zero. Alarms are now counted only inside `[onset, onset+duration+30]`, the same window the rest of the evaluation uses.
2. **Pre-existing alarms.** Even inside the window, an alarm that the unmodified series produces anyway (a heat wave, a persistent shift, a false alarm of the drift detector) cannot be attributed to the injection. The clean-series event mask is subtracted from the attacked-series mask.
3. **Resolution.** A 0.05 σ bisection tolerance cannot distinguish "no margin" from "0.05 σ of margin", and the returned lower bound was silently 0 whenever the last step alarmed. The tolerance is now 0.01 σ.

With those three fixed the black-box attacker reaches 0.05–0.9 σ of undetected bias depending on site (Table 6.4), which is the interpretation the theory predicts: below the detectors' effective resolution an adversary is free, above it it is not, and the width of the free band is site-dependent because the residual scale, the AR order and the false-alarm transfer differ by site.

The evasion *curve* had a fourth artefact: a single injection window per budget makes the curve non-monotone, because the achievable undetected bias depends on which days the window covers (a calm spell hides more than a stormy one). The curve now averages four windows $\times$ both signs per (site, budget), and both monotonicity and the L3 gap survive that averaging.

3h. Coverage: pairs that cannot be modelled are named, not hidden

KHARGA precipitation has 13 valid days in eleven years, and one earlier run wrote an `ERROR` row into a results table for it. A results table is not the place for an exception message, and quietly dropping the row is worse: the reader cannot tell a missing pair from a pair that failed. `results/excluded_pairs.csv` now lists every excluded (site, variable) pair with the reason and is regenerated on every evaluation run; the tables cover the 34 design pairs, which is all of them on the current bundle. The exclusion path is exercised by `tests/test_reports.py`.

3i. The blind spot no statistic can cover: D7, the availability channel

Every detector in L2 answers "does this value agree with an independent description of the same day". All of them are blind to the failure in which the value stops arriving: a dropout makes the innovation NaN, so nothing exceeds a threshold, and a silenced station is indistinguishable from a quiet one. In a false-data- injection exercise that is the most important gap of all, because withholding the record is a data-integrity attack that has no statistics behind it.

D7 therefore tests the reporting cadence, with a data-derived envelope: a run of missing days longer than 1.5x the longest gap observed in the calibration period is an outage fault. It is a rule rather than a calibrated test (there is no continuous statistic whose tail defines a budget), and it is deliberately kept out of the statistical alarm stream, so that a station with a broken modem cannot consume the detectors' false-alarm budget. The measured consequence in Table 7 is that 12 % of the bundled days are missing and a quarter of the pairs have at least one outage event in the test period — i.e. the channel fires on real gaps, not on noise.

Two attribution bugs were fixed while making the diagnosis numbers honest, and both are the same mistake in different places: **the first alarm of the record is not the alarm of the injection.** The diagnosis experiment used to take `events[0]` over the whole series, so every row of the confusion matrix was a diagnosis of some unrelated false alarm six months earlier — which is why the matrix looked identical for every injected family. It now diagnoses the first exceedance inside `[onset, onset+duration+30]`, preferring the exceedance day over the cooldown-collapsed event onset so that an injection landing within 14 days of an earlier alarm still gets its own evidence. The generator had the mirror-image bug:

`make_spec` ignored the requested magnitude for the families parameterised by an amplitude, so ”inject a 12 sigma spike” injected the configured 1 sigma default, and the spike row of every experiment was measuring noise. Both are covered by tests.

Data sources

The specification marks several items **[Verify]** (section 18). This note records each one, the documentation or probe that resolved it, and the coding consequence. Everything here was checked against the live services in October 2026 while the bundle was built; the bundle itself is committed so that a reader never needs the network.

1. NCEI data-service parameter names — resolved

Endpoint actually used:

```
https://www.ncei.noaa.gov/access/services/data/v1
    ?dataset=daily-summaries
    &stations=<11-char GHCN id>          (comma-separated for several)
    &startDate=YYYY-MM-DD&endDate=YYYY-MM-DD
    &dataTypes=TMAX,TMIN,PRCP
    &format=csv&units=metric&includeAttributes=true
```

Verified behaviour (probed on CARTHAGE `TSM00060715`, 2015-2024):

- **Units**: `units=metric` is required for °C and mm; the default is US units.
- **QC flags**: with `includeAttributes=true` every variable gets a companion column `<VAR>_ATTRIBUTES` whose comma-separated fields are `source, qflag, ...`. We keep a value only when the `qflag` field is empty (GHCN convention: empty = passed all QC); flagged values are *not* silently repaired, they become missing days. This is the difference between "the archive says this value is doubtful" and "the sensor is lying", and the pipeline must not confuse the two.
- **Missing days are omitted from the CSV, not filled** — so the calendar has to be rebuilt explicitly (`io.load_pair` reindexes onto a complete daily calendar and sets a gap flag). Treating the CSV row order as the calendar is a classic and silent source of misalignment.
- **AWND is not populated** for the Mediterranean stations in this bundle, so the wind variable was dropped early (the specification allows a subset); only `TMAX`, `TMIN`, `PRCP` are used.
- Station identifiers are the 11-character GHCN ids (e.g. `TSM00060715`), obtained from `ghcnd-stations.txt` and cross-checked against `ghcnd-inventory.txt` for the first/last year of each element (that check is what selected the sites in `config/sites.yaml`, and it is re-runnable as `python -m tsfdi sites`).

2. NASA POWER missing-value marker and parameter names — resolved

Endpoint actually used:

```
https://power.larc.nasa.gov/api/temporal/daily/point
    ?parameters=T2M,T2M_MAX,T2M_MIN,RH2M,WS2M,PRECTOTCORR,ALLSKY_SFC_SW_DWN,CLRSKY_SFC_SW_DWN
    &community=AG&latitude=<lat>&longitude=<lon>
    &start=YYYYMMDD&end=YYYYMMDD&format=JSON
```

- **Missing marker is `-999.0`** (fill value); it is mapped to `NaN` at load time and *never* to zero — for precipitation that distinction is the whole game.
- Values are a nested dictionary `properties.parameter.<NAME>.<YYYYMMDD>`, with string keys, so the date index must be parsed rather than positional (another silent-misalignment trap: a POWER response is not a rectangular array).
- **Timestamps are local solar time (LST)**, not UTC. GHCN-Daily days are local calendar days, so both roads are treated as local days and the *lag* between them is estimated on the calibration period instead of being assumed to be zero. Measured lags are 0 or ±1 day for the bundled stations, consistent with LST versus local-standard-time reporting.
- `PRECTOTCORR` is a corrected precipitation flux; it is the closest daily analogue to a gauge total, and its correlation with a point gauge is much weaker than for temperature — the reason precipitation carries a restricted claim in the report.

3. Hourly station data — available, deliberately not used

An hourly source does exist and was checked: NCEI's **Integrated Surface Database (ISD / "Global Hourly")** is being replaced by **GHCNh (Global Historical Climatology Network hourly)**, with GHCNh using the same station identifiers as GHCN-Daily (NCEI announcement, service-change notice). The transition is date-dependent in the NCEI data-access API: `global-hourly` for dates before 2025-01-01 and `global-historical-climatology-network-hourly` from 2025-01-01 onwards, with **UTC timestamps** for hourly queries (unlike the daily service).

Decision: **daily data is used for the evaluation**, and the architecture is hourly-capable (the streaming pipeline is O(1)-state per reading and provenance costs ~0.2 ms/record). The reasons are stated rather than implied: (i) the challenge's attack magnitudes are defined in units of the seasonal range and the sigma of the *daily* residual, so an hourly evaluation would have needed a second calibration protocol; (ii) GHCNh coverage for the North-African stations in this bundle is sparse, which would have silently changed the site set; (iii) the reference product is daily, so an hourly evaluation would

have required an interpolation of the reference that would then dominate the residual. This is a documented scope decision, not an oversight.

4. The 12 % / 15 % / 7-sample profile and how a percent is defined — resolved by documentation

The specification leaves the meaning of ”12 %” open. The implementation uses, and every table states:

- **temperature**: percent of the site's *seasonal range* (max − min of the fitted seasonal baseline). Percent of a Celsius value is meaningless — 12 % of 5 °C is 0.6 K and 12 % of 30 °C is 3.6 K for the same station in the same winter — so the range is the only scale that is stable across sites and seasons.
- **precipitation and wind**: a multiplicative factor (12 % more rain), because the working coordinate is already a variance-stabilised transform of a non-negative quantity.

Every table also reports the **σ-equivalent** of each percent, computed from the site's own residual scale, so a reader who prefers σ units never has to convert.

5. Jury VM specification — unknown, therefore assumed worst case

Unknown at the time of writing. Consequences that are implemented, not promised:

- no network at run time (bundle committed; `make all` never fetches);
- no GPU and no compiler (pure NumPy/SciPy/pandas wheels, pinned in `requirements.txt`);
- the dashboard is optional: `python -m tsfdi report-static` writes a self-contained HTML file with inline SVG and no JavaScript, and the CLI reports everything the dashboard shows;
- the model cache is deterministic for a given seed and manifest, so a re-run either reproduces `expected_results.json` within tolerance or fails loudly.

6. Reproducing the bundle

`data/bundle/FETCH_INFO.json` records the exact request strings, retrieval timestamps and row counts per file. `python -m tsfdi fetch --force` rebuilds it. The archive providers revise their holdings, so a fresh fetch may legitimately change a handful of values; that is precisely why the bundle is pinned and signed rather than fetched on demand.

State of the art

The challenge sits at the intersection of three literatures. This note records what each provides, what it assumes, and which assumption breaks for in-situ climate sensors — the last column is the reason for each design choice in the system.

Approach	Representative work (line of research)	Contribution	Assumption that fails here	What we take / change
False-data injection in control systems	Secure state estimation and stealthy-attack analysis for power and water networks (Liu, Ning, Mo, Sinopoli and the follow-on literature)	Formal adversary model; the notion of a *stealthy* attack that keeps every residual below its threshold; observability conditions for detection	The plant is a linear model with known inputs and full measurement redundancy; observations are Gaussian and complete	We adopt the adversary hierarchy (A0–A3) and the *damage-under-a-budget* framing; we replace the linear state estimator with a robust regression on an independent reference, and replace ”detectable/unobservable” with a calibrated false-alarm budget
Climate data quality control	GHCN-Daily QC flags, BSRN quality tests, neighbour-comparison and spatial-consistency checks	Operational, decades-proven fault detection (range, internal consistency, persistence, spatial outliers)	The checks detect *implausible* values; a falsified value that mimics weather passes all of them	We keep a subset of them as D4 (single-sample excursions) and D7 (reporting-gap availability) but add the reference-anchored residual tests that a plausible falsification must violate
Change-point and sequential detection	Page’s CUSUM, Roberts’ EWMA, PELT/binary segmentation, matched filtering	Optimal (in the Lorden sense) detection of a shift for a given false-alarm rate; exact inference for change points	Independent, stationary, complete, Gaussian-ish residuals	We use CUSUM/EWMA/PELT as D1/D2/B2 after *whitening, degapping and bias/scale stabilisation*; every threshold comes from a block bootstrap (or a phase-randomised surrogate where the bootstrap fabricates the signal)
Residual-based sensor validation with redundancy	Analytical-redundancy literature, cross-sensor validation	Cheap and powerful when several sensors measure the same quantity	A single station measures the point; no second sensor exists at the same location	We use a *different kind* of redundancy: an independent gridded product (reanalysis) that is cheap, complete and public, at the cost of site-representativeness error
Replay / adversarial-sequence detection	Anti-spoofing for GPS and sensor networks; subsequence matching	Detection of copied sequences via cross-correlation and exact matching	The signal is high-entropy and not strongly autocorrelated	We match on the *anomaly* road (a level series repeats its own annual cycle), exclude degenerate windows (dry spells are self-similar), and pair the match with a reference-contrast test
Provenance and integrity	Merkle hash chains, digital signatures, trusted execution	Tamper evidence for data in transit and at rest	A signature says nothing about truthfulness	L0 implements the chain and *explicitly* limits its claim; A3 is the case it cannot cover
Adaptive/ evasive adversaries	Urbina et al. on stealthy sensor attacks; game-theoretic detection	Quantifies what an adversary achieves under a detection constraint	Requires knowing the detector’s objective and its budget	We implement A2 white-box and black-box, and report damage at a fixed budget rather than a single detection rate

The gap, stated precisely

No published pipeline, to our knowledge, (a) anchors a point sensor to a **hash-pinned independent reanalysis reference**, (b) calibrates **every** threshold to a false-alarm budget in the presence of serial correlation, 35–45 % missing days and a drifting station/reference relationship, and (c) reports detectability, false alarms and **attacker damage** on the same footing, with the failure modes of the statistics — not only the success modes — measured on real archives. This report does all three, and pays for it by documenting the places where the result is a limitation rather than a win.

References (line of research, not a bibliography of one paper)

- Secure state estimation / stealthy attacks: Liu, Mo & Johansson; Pasqualetti, Dörfler & Bullo; Teixeira et al.; Urbina et al.
- Sequential detection: Page (1954); Roberts (1959); Lorden (1971); Basseville & Nikiforov; Killick, Fearnhead & Eckley (PELT); Truong, Oudre & Vayatis (`ruptures`).

- Robust regression and scale: Huber; Rousseeuw & Croux (MAD/SN).
- Spectral and surrogate methods: Welch; Theiler et al. (phase randomisation); Ledoit & Wolf (shrinkage covariance).
- Climate data records and QC: Menne et al. (GHCN-Daily); Dürr et al. / Ohmura et al. (BSRN); reanalysis as a reference product: Hersbach et al. (ERA5); POWER project documentation (NASA LaRC).
- Reproducibility: Sandve et al.; Piccolo & Frampton (offline, container-based reproduction); `pytest` and `make`-based pipelines.

Reproducing every number

Everything needed is in this repository. The data bundle is committed and hash-pinned, so no command in the default pipeline touches the network.

1. Environment

Target: a fresh Ubuntu 22.04/24.04 machine, 4 GB RAM, 2 vCPU, no GPU, no compiler toolchain. Python $\geq$ 3.11.

```
python3 -m venv .venv && . .venv/bin/activate
pip install -r requirements.txt        # pinned versions
```

`requirements.txt` pins every runtime dependency (numpy, pandas, scipy, scikit-learn, PyWavelets, cryptography, matplotlib, PyYAML, ruptures, tabulate). Three optional files add what the runtime does not need:

File	Contents	Needed for
`requirements-dev.txt`	`pytest`	`make test`
`requirements-app.txt`	`streamlit`	`make dashboard`
`requirements-report.txt`	`pypdf`	`make report-pdf` (plus XeLaTeX, below)

`pyod` is not required: the Isolation-Forest baseline B3 uses `scikit-learn`'s implementation when `pyod` is absent. The dashboard is not required either — `results/static_report.html` is self-contained and needs no Python at all. Nothing else is imported.

2. One command

```
make all          # data-verify -> gonogo -> calibrate -> experiments -> report -> verify -> static
```

`make all` is a wrapper over the CLI; each stage can be run alone and writes its own artefacts:

Target	Command	Output
`make data-verify`	`python -m tsfdi data-verify`	verifies the signed manifest
`make gonogo`	`python -m tsfdi gonogo`	`results/gonogo.{csv,md}`
`make calibrate`	`python -m tsfdi calibrate`	`results/calibrate_all.log` (not committed), model cache
`make experiments`	`python -m tsfdi experiments`	`results/raw/*.csv` (tidy, one row per cell)
`make report`	`python -m tsfdi report-figures` then `report-text`	`results/tables/*.csv/.md`, `results/figures/*.png`, `docs/report.md`
`make report-pdf`	`python -m tsfdi report-pdf`	`docs/report.pdf`, `docs/appendix.pdf`, `docs/documentation.pdf` and their `.tex` sources (XeLaTeX)
`make verify`	`python -m tsfdi verify`	exit code 0/1 and the metric-by-metric diff on stdout
`make bench`	`python -m tsfdi bench`	`results/bench_provenance.{csv,json}`
`make test`	`python -m pytest tests/`	unit and integration tests (81, no network; needs `requirements-dev.txt`)
`make dashboard`	`streamlit run src/tsfdi/app/dashboard.py`	interactive demo
`make static`	`python -m tsfdi report-static`	`results/static_report.html` (no JS, self-contained)

The PDFs need XeLaTeX and a handful of packages, none of which the pipeline imports: `xelatex` with `booktabs`, `longtable`, `titlesec`, `enumitem`, `caption`, `fvextra` and the `tex-gyre` fonts (`texlive-xetex` + `texlive-latex-recommended` + `texlive-fonts-recommended` on Debian/Ubuntu, or `tlmgr install booktabs titlesec enumitem caption fvextra tex-gyre microtype` on a minimal TeX Live). The committed PDFs are the artefacts of record; re-running the target is only needed if the Markdown changes.

`bash run.sh` does the same once the environment above is installed (it does not create the virtual environment itself), and accepts `--quick` for a smoke run of the whole pipeline in a few minutes.

3. Determinism

- One seed: `config.SEED` (`src/tsfdi/config.py`). Every bootstrap replicate, phase-randomised surrogate, injection and train/test split derives from it.
- The (site, variable) models are cached in `results/cache/models`; deleting the directory and re-running `make calibrate` reproduces the same thresholds to bit level on the same data manifest.
- `make verify` compares the metrics computed from the current `results/` tree with `expected_results.json` at the repository root. Because `make all` regenerates `results/` first, the comparison is a genuine reproduction check; run on its own against the committed results it only confirms internal consistency, which is why it is documented as the last step of the chain. Rate metrics are compared with an absolute tolerance (a rate of 0.02 alarms/sensor-year cannot be compared with a 2 % relative tolerance), proportions with a 0.02 absolute tolerance.

4. Data

`data/bundle/` contains 24 CSVs (12 stations × up to 3 variables) plus `data/bundle/FETCH_INFO.json`, `MANIFEST.sha256` and its Ed25519 signature `MANIFEST.sig`. The reference files came from NASA POWER (`/api/temporal/daily/point`, community AG); the sensor files from the GHCN-Daily access API (`dataset=daily-summaries`, metric units, quality flags kept). `data/bundle/FETCH_INFO.json` records the exact request strings, the retrieval date, and the row counts, so the bundle can be rebuilt from scratch if a network is available:

```
python -m tsfdi fetch --force
```

Re-fetching is **not** part of `make all`: the providers revise their archives, and a reproducibility claim that depends on a live API is not a reproducibility claim. The pinned bundle is the artefact of record.

5. Runtimes on the reference machine

Measured on 2 vCPU / 4 GB (no GPU). Figures are wall-clock and exclude the one-off data fetch.

Stage	Runtime
`data-verify`	< 1 s
`gonogo` (34 pairs, incl. Monte-Carlo minimum detectable bias)	see `results/gonogo.md` header
`calibrate` (34 pairs, 14 detectors, 4 budgets)	≈ 28 min
`experiments` (full grid)	≈ 5.6 h single-core; the shipped run used the split below
`scripts/run_grid.py results/raw/grid_tmax.csv TMAX 20`	≈ 1.3 h (one core)
`scripts/run_grid.py results/raw/grid_secondary.csv TMIN 10` then `PRCP`	≈ 0.4 h + 0.2 h (second core)
`scripts/run_rest.py` (profile, FA, diagnosis, baselines, ablation, efficiency, evasion)	≈ 0.9 h
`report-figures`	< 30 s
`bench` (100 000 records)	< 60 s
`tests`	see `make test`

The generalisation grid is the only stage that does not fit comfortably in one process on the reference machine, so it is split by variable and run two at a time:

```
python3 scripts/run_grid.py results/raw/grid_tmax.csv TMAX 20      # core 1
python3 scripts/run_grid.py results/raw/grid_secondary.csv TMIN 10 # core 2, then PRCP 10
python3 scripts/merge_grid.py            # -> results/raw/table2_grid_raw.csv
python3 scripts/run_rest.py              # everything except the grid
```

`scripts/README.md` documents this parallel path and the constraint behind it: a change to how a family's magnitude is calibrated invalidates that family's rows and nothing else, so the suite is re-run per family rather than wholesale.

The dominant cost is the bootstrap calibration, which is O(replicates × days × detectors) per pair and was deliberately traded against Monte-Carlo error (see `docs/design.md` §5).

6. Repository layout

```
config/          YAML configuration: sites, detectors, attacks, profile, decisions
data/bundle/      pinned CSVs + signed manifest (the artefact of record)
docs/             threat model, design decisions, this file, report
results/          every generated artefact (raw, tables, figures, cache)
scripts/          optional parallel entry points (see scripts/README.md)
```

```
src/tsfdi/       the package (sp, detect, attacks, provenance, diagnose, eval, app)
tests/           unit and integration tests (pytest)
Makefile         the seven stages above
run.sh           the same, from a prepared shell
```